Seeing the Crowd Coming: An OLTP-to-Lakehouse Pipeline for National Park Tourism Demand

Chujie Wang | Final project proposal | October 2026

1. The problem and who cares

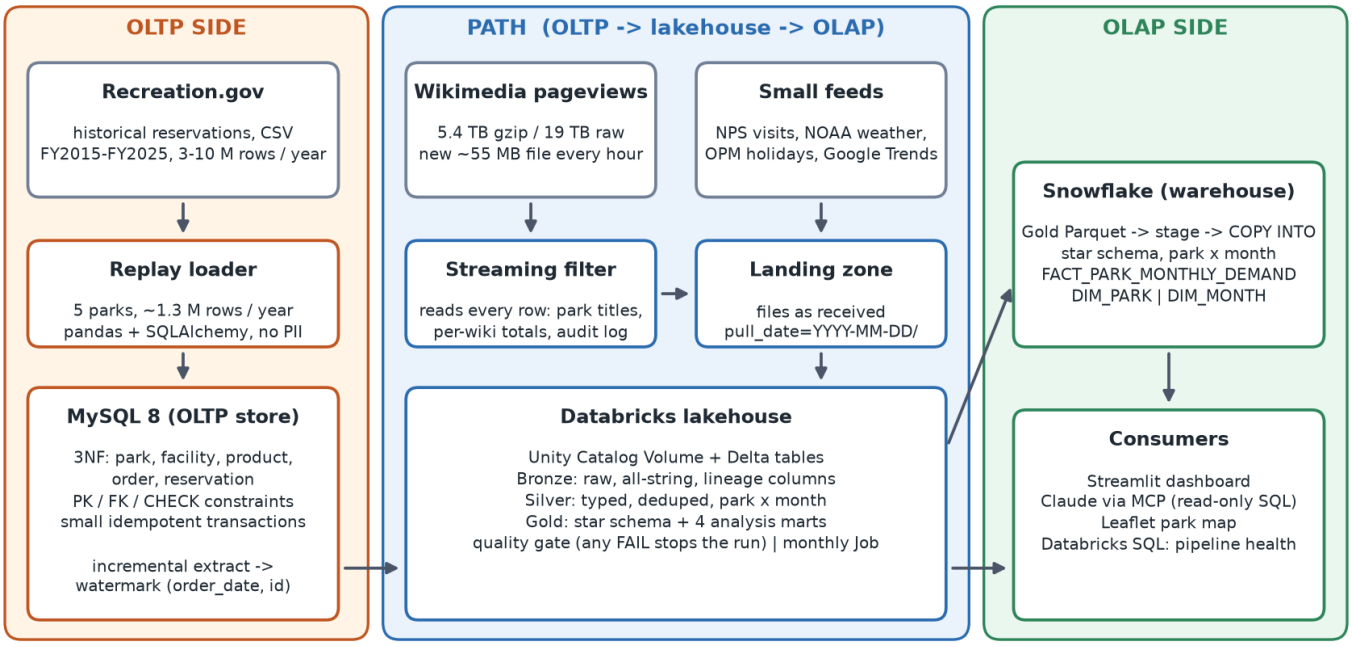
Demand at America's busiest national parks now outruns capacity at peak times. In FY2024 alone, Recreation.gov logged 311,293 Yosemite timed-entry reservations and 190,008 Zion lottery-permit records (applications included), and Yellowstone's June 2022 flood closure cut June visits 44% below June 2021. Yet the official count arrives late: NPS treats current-year figures as preliminary and publishes most data only through the last calendar year (in October 2026 its API still returns no 2026 months). **Question: beyond the calendar, which signals tell us one to three months ahead how busy each park will be, and how much of a month's swing is seasonality versus shocks?** Candidate signals: reservations already on the books, online attention, search interest, weather and holidays. **Who cares:** park managers (staffing, shuttles, permit quotas), gateway-town businesses (lodging, guides, restaurants) and travelers choosing when to go. **Scope:** five parks (Yellowstone, Yosemite, Grand Canyon, Zion, Great Smoky Mountains), park x month grain, 2015-2025.

2. The dataset: where it lives, our rights, and the data rule

Source	Role	Where it lives	Rights	Shape and size (measured Oct 2026)
Wikimedia hourly pageviews	Attention; our 5 TB+ dataset	dumps.wikimedia.org/other/pageviews	CC0 (all Wikimedia Analytics datasets)	One gzip file per hour since May 2015 (100,597 files), 35-49 GB per month. About 5.4 TB compressed (4.9 TiB), about 19 TB uncompressed, about 600 billion rows. One sample hour: 52.8 MB, 6.04 M rows.
Recreation.gov historical reservations	Booking intent; the OLTP side	ridb.recreation.gov bulk download, one CSV per fiscal year (split by use date)	Free federal data under the RIDB API Access Agreement (accepted on download); credit "data source: ridb.recreation.gov"; customer ZIP only, no names	FY2015-FY2025: about 3.5 GB zipped, about 65 M rows. FY2024: 9.78 M rows, 3.9 GB unzipped, 35 columns; 1.29 M rows in our five parks.
NPS visitation, NOAA weather, OPM holidays, Google Trends	Outcome and context	irmaservices.nps.gov API; ncei.noaa.gov API (Global Summary of the Month); opm.gov tables (scraped); trends.google.com via pytrends	NPS, NOAA, OPM: U.S. government works, public domain. Trends: reuse with citation under Google's ToS; pytrends is unofficial	660 park-months of visits; 1,257 station-months from 10 stations; 172 holiday rows (2015-2030); 5 parks x 132 months of search interest, one request per park.

How it meets the data rule. The Wikimedia hourly dumps are our 5 TB+ dataset. **Size:** about 19 TB uncompressed (5.4 TB gzip) and about 600 billion rows since May 2015, beyond a laptop's disk or memory. **Velocity:** a new six-million-row file lands every hour (about 0.5 TB of gzip a year), so ingestion must be incremental. A streaming filter reads every row: it keeps the five park articles (six million rows in, about ten out) and totals views per wiki per hour, a denominator that removes Wikipedia's site-wide traffic trend from park attention. An audit table logs bytes and rows read per file, and its totals prove the full read. Recreation.gov gives the OLTP side a real transactional workload. Nothing here relates to Sakila or ClassicModels.

3. Target architecture



OLTP side: MySQL plays the booking system's operational database (the system of record). The loader replays the five parks' reservations in order-date sequence as small, idempotent transactions into a 3NF schema that enforces keys and constraints, and it drops vendor test rows and donations. **Path:** an incremental extract (watermark on order date plus reservation id) writes only new rows to Parquet. These

files and the landed source files are uploaded to a Unity Catalog Volume (a governed file area in Databricks), so Databricks never needs a network path into MySQL. A monthly Databricks Job builds Bronze (raw text plus lineage), Silver (typed, deduplicated, park x month) and Gold (star schema and analysis marts), and a quality gate stops the run on any failure; the Wikimedia worker runs hourly. **OLAP side:** Gold is exported as Parquet, PUT to a Snowflake stage and loaded with COPY INTO into the park x month star schema, which serves the Streamlit dashboard and a read-only SQL tool for Claude via MCP (Model Context Protocol). The Databricks SQL dashboard only monitors runs and quality checks. **Status:** the four small feeds are ingested, and a Bronze/Silver prototype, run with Spark on the real files, returns 660 park-months with 0 duplicate keys and 99.8% temperature coverage. Wikimedia and Recreation.gov are not yet ingested; they are exactly what A1 and A2 test.

4. The two riskiest assumptions and how we test them by Week 8

A1 is the risk to the data rule; A2 is the risk to the question.

A1. We can read the full 5.4 TB history once, on free resources, by filtering while streaming.

- **Why it is risky:** filtering is cheap, since one hourly file decompresses and filters in about 2 s on a laptop. Transfer is not. dumps.wikimedia.org rate-limits downloads and allows 3 connections per IP, and at the 5.2 MB/s we measured on one connection the full history takes about 12 days. Keeping up afterwards is easy (one new file takes about 10 s). Databricks Free Edition limits outbound internet to an allow-list and shuts compute down for the day when a quota is exceeded, so the worker may have to run outside Databricks.
- **Test by Week 8:** process one full week (168 files, 9-10 GB) with 3 parallel streams and a compliant User-Agent. Log MB/s, rows/s and any lost files, and check whether serverless compute can reach the site. Reconcile daily park totals with Wikimedia's per-article REST API (agent=user, all-access). Files are named for the end of their hour, so UTC day D is the files stamped D 01:00 through D+1 00:00. Target: an exact match. In a spot check, one hour's English Wikipedia totals already matched exactly.
- **Decision rule:** pass if 3 streams sustain at least 12 MB/s (full history in about 5 days) with no lost files and an exact match; the resumable backfill then starts at once. If slower, we split the years across two workers (the laptop and a free-tier cloud VM). We do not drop years, because the data rule rests on the full read. Until the backfill finishes, the analysis uses the REST API's monthly per-article counts, which are built from the same data.

A2. The leading signals can be aligned to park x month, and they lead visitation beyond the calendar.

- **Why it is risky:** reservations are keyed by use date and facility, not park-month, and the files drift: 56 columns in FY2015-FY2017, 57 in FY2018, 36 in FY2019 and 35 from FY2021. Date formats change, and the park name moves from RegionDescription to ParentLocation in FY2019. FY2018, the last year before Recreation.gov's October 2018 vendor change, ships with a README warning that it "has not been fully verified for accuracy and completeness." In a sample of FY2024 rows the median booking lead time is only 0-22 days, and where timed entry caps entry, visitation partly reflects the quota rather than demand. One Google Trends request for all five parks left 61% of values at 5 or below, so we pull each park separately (comparable within a park only). And each park has only 132 months; April 2020 was closed or nearly so in all five.
- **Test by Week 8:** load FY2015-FY2025 into MySQL with every foreign key passing, and build the 660-row integrated table with 0 duplicate keys (booking signals stop in September 2025 until FY2026 is published; attention starts in May 2015). Compute the reservations on the books for month m at the end of month m-1. Apply the same seasonal adjustment to visitation and to every signal (each month divided by that park's average for that calendar month, closures excluded). Then correlate at lags 0-3, where lag k means the signal is known k months before the visitation month.
- **Pass and fallback:** pass if the same signal reaches r >= 0.5 at a lag of 1 or more in at least three parks, and still does with 2020 excluded. If older files cannot be mapped to parks, we keep the fiscal years that map cleanly and report the shorter window. If no signal passes, we reframe the deliverable as explaining demand (seasonality, weather, shocks) and report the negative forecasting result.

5. Team roles

Solo project. Chujie Wang covers every role: **data engineer** (ingestion workers, MySQL OLTP schema and replay loader, incremental extract), **lakehouse engineer** (Databricks Bronze/Silver/Gold, quality gate, Job), **analytics engineer** (Snowflake star schema, Streamlit and Databricks dashboards) and **analyst** (Week 8 tests, findings, README, presentation). Because one person builds everything, scope is fixed at five parks and park x month; more parks or a daily grain are stretch goals.